

Computer Architecture Study

Master:

Dr. Safari

Project:

Single-Cycle Micro-Arch Design for RISC-V ISA

Contributors:

Anita Zekavat & Amir Hosseini

Project Workflow

1. RTL Elements Identification

Begin by identifying and designing the necessary RTL components for the project, including memory and logic elements.

2. Data Path

Next, construct the **Data Path** by interconnecting the RTL elements to enable proper data flow and processing within the system.

3. Controller

Design the **Controller** responsible for generating control signals that coordinate the operations of the **Data Path**.

4. Top Module

Finally, design and describe **Top Module**; Then test it by a **sort algorithm**, in **assembly**.

RTL Elements

At first we defined our Elements, which doesn't need any special design, so we just describe them by **Verilog**.

There are some basic modules such as **AND**, **REG**, **MUX**, **etc.** available at:

`"$PATH/TO/PROJECT/"code/elements/`

And abstractive modules of memory, **DATA_MEM**, and **INST_MEM**, available at:

`"$PATH/TO/PROJECT/"code/mem/`

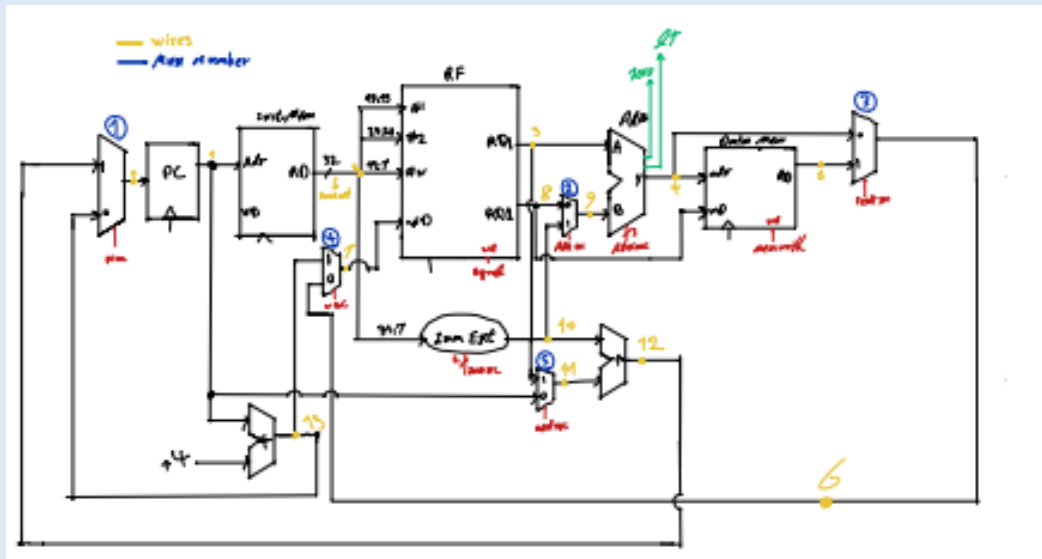
Data Path design

Design:

At first we start with designing top module **Data Path**. We must make a schematic design.

The design of **DP** (Data Path) is available at:
"\$PATH/TO/PROJECT"/design/papers/

Also you can see down here:



DP schematic design

Wires' and MUXs' Indexes in Verilog description are entirely as same as the picture above, also Wire[6] is named "**InstOut**", which is noticeable.

Also **ALU**, and **Imm Ext** must be designed based on our needs; Their function table is available at:
"\$PATH/TO/PROJECT"/design/tables/

The ALU has **zer** signal, Active when **Y = 0**, and **lt** signal active when **A < B**; **lt** signal is "**don't care**" when ALU function is **not** on **slt**.

Code:

After design, the schematic diagram, and function tables are described by Verilog HDL, which is available at:

`"$PATH/TO/PROJECT"/code/data_path/`

Also a test bench is generated to test its' functionality, available at:

`"$PATH/TO/PROJECT"/code/test/`

we expected that the hardware execute a simple **DPTTest** assembly code available at:

`"$PATH/TO/PROJECT"/assembly/DPTTest/`

Simulation:

We designed an experiment to find out whether data path is doing well or not, we put two numbers in memory and then ask our **Data Path** to sum these two numbers, if the answer is more or equal that 10, put 1000 in memory cel **0x8**, else put 0 on memory cell **0x8**; Then by observing data on memory cell **0x8** we can find out whether data path is correct in function or not.

We make three experiments to test the design, the result is available down here:

a = 1, b = 10000000000000001:

the result can be measured by observing data in memory and register file, we have:

[illegible]

Data in memory cells (captured from "Model Sim")

[illegible]

Data in registers (captured from "Model Sim")

a = 11, b = 11111100000000001000000000001:

You can see the difference with previous:

00000000	000000000000000000000000000000000011	111111000000000000100000000000001	000000000000000000000000000000000000
00000003	000000000000000000000000000000000000	000000000000000000000000000000000000	000000000000000000000000000000000000
00000006	000000000000000000000000000000000000	000000000000000000000000000000000000	000000000000000000000000000000000000
00000009	000000000000000000000000000000000000		
0000000c			

Data in memory cells (captured from "Model Sim")

[illegible]

Data in registers (captured from "Model Sim")

a = 11111011100000000000000001101 ,

b = 000001000111111111111111111101:

As $a + b = 10$, it is an edgecase. We have:

```
00000000 111111011100000000000000000000001101 00000010001111111111111111111101 000000000000000000000000111101000
00000003 000000000000000000000000000000000000 000000000000000000000000000000000000 000000000000000000000000000000000000
00000006 000000000000000000000000000000000000 000000000000000000000000000000000000 000000000000000000000000000000000000
00000009 000000000000000000000000000000000000 000000000000000000000000000000000000 000000000000000000000000000000000000
0000000c
```

Data in memory cells (captured from "Model Sim")

00000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000003	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000006	00000000000000000000000000000000	00000000000000000000000000000000	11111011100000000000000000000000
00000009	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
0000000c	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
0000000f	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000012	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000015	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000018	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
0000001b	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
0000001e	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000021			

Data in registers (captured from "Model Sim")

These simulations show that data path is doing well in functionality, it was better to test more cases but we skip this part due to lack of time in project deadline.

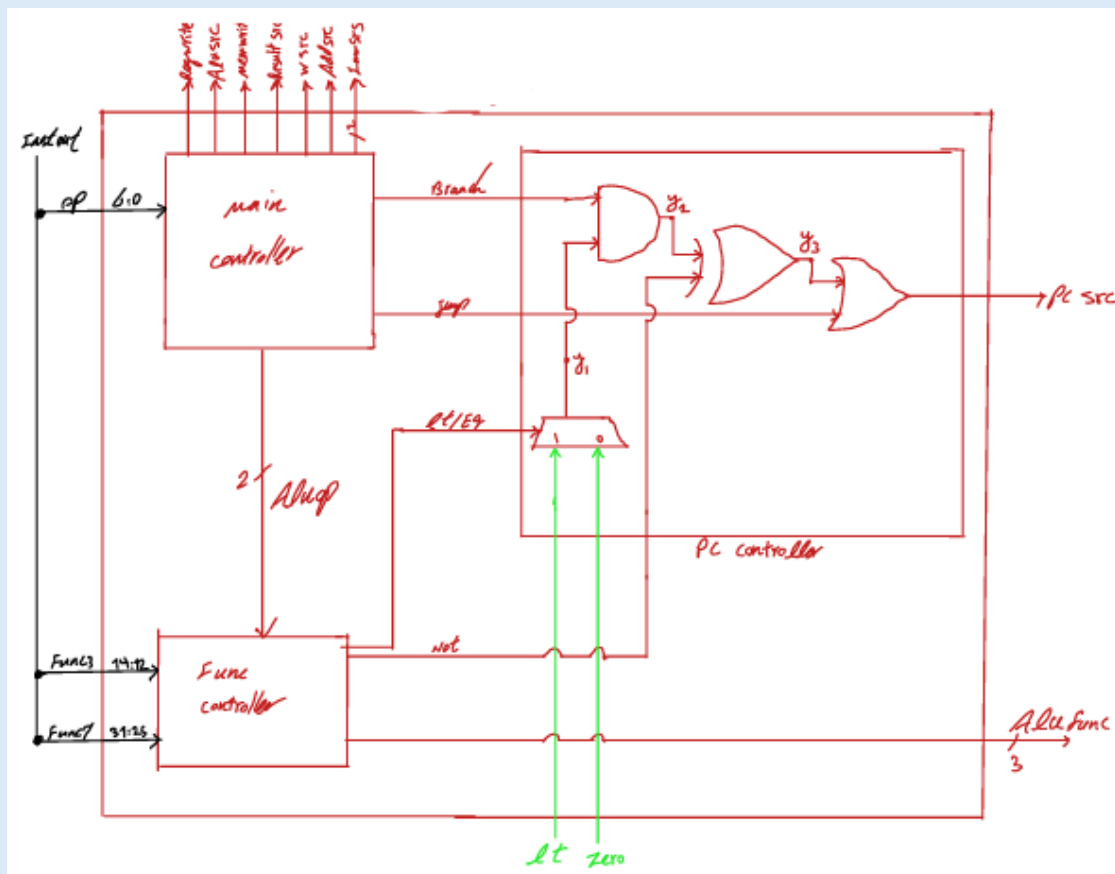
Controller Unit:

Design:

We implemented “**Controller Unit**”, using three modules: “**Main Controller**”, “**Func Controller**”, and “**PC Controller**”.

The interconnections' schematic diagram is available at: ["\\$PATH/TO/PROJECT"/design/papers/](#)

Also you can see the schematic diagram down here:



Control Unit Schematic Design

Also, the interconnections of **PC Controller** is obviously shown in the diagram above.

Now we explain two other modules in the next page, **Main Controller**, and **Func Controller**.

Main Controller:

It decodes the **op**, available in **Inst.** code to control signals for **Data Path**; Also gives an **ALU Op** signal, used in “**Func Controller**”, and gives **Branch** and **Jump** signal used in “**PC Controller**”.

The truth table is designed, which available at: “\$PATH/TO/PROJECT/”tables/

Also you can see the table down here, some columns are written in Decimal Base, due to readability:

Inst	op	ALU Op	RegWrite	ALUSrc	Branch	Jump	MemWrite	ResultSrc	WSrc	ImmSrc	AddSrc
R-T	51	10	1	0	0	0	0	0	0	–	–
B-T	99	01	0	0	1	0	0	–	–	10	0
I-Arithmetic	19	11	1	1	0	0	0	0	0	00	–
lw	3	00	1	1	0	0	0	1	0	00	–
sw	35	00	0	1	0	0	1	–	–	01	–
jal	111	–	1	–	–	1	0	–	1	11	0
jalr	103	–	1	–	–	1	0	–	1	11	1

Main Controllers' truth table

Func Controller:

It takes **func3**, **func7** from the **Inst.** code, along with the ALU Op signal from “**Main Controller**”, and then generates the appropriate control signals for the **ALU** and the “**PC controller**”.

The truth table is designed, available at:
“\$PATH/TO/PROJECT”/tables/

Also you can see the table down here, some columns are written in Decimal Base, due to readability:

ALU Op	Func3	Func7	Inst	ALU func	lt/eq	not
00	–	–	lw/sw	000	–	–
01	0	–	beq	001	0	0
01	1	–	bnq	001	0	1
01	4	–	blt	101	1	0
01	5	–	bge	101	1	1
10	0	0	add	000	–	–
10	0	32	sub	001	–	–
10	7	0	and	010	–	–
10	6	0	or	011	–	–
10	2	0	slt	101	–	–
11	0	–	addi	000	–	–
11	4	–	xori	100	–	–
11	6	–	ori	011	–	–
11	2	–	slti	101	–	–

function table of Func Controller

Notice:

The column “**Inst**” is not a part of truth table, but defined for better understanding, and less bug probability.

Code:

Each module is described by Verilog HDL, available at:

“\$PATH/TO/PROJECT/”code/controller/

Notice:

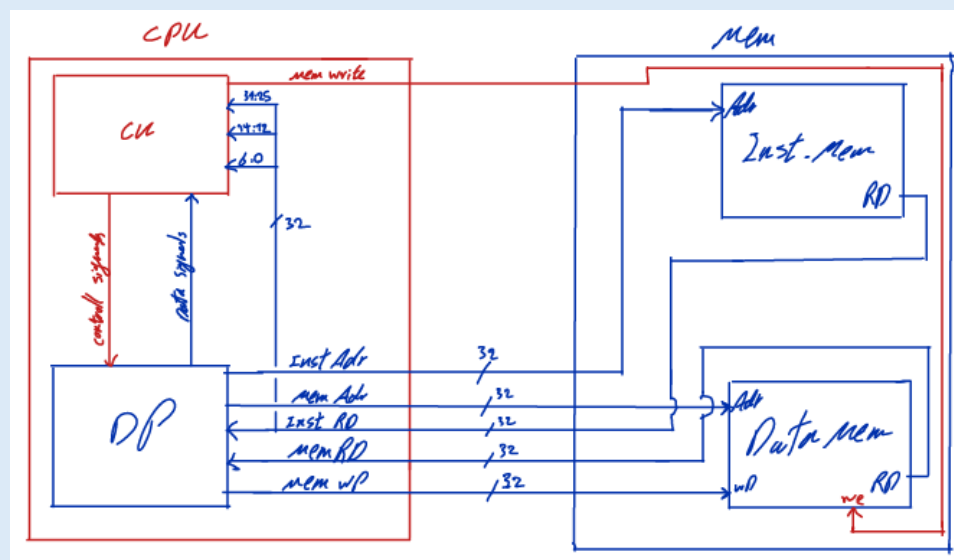
We haven't designed any test scenario for this part, The whole system will be tested in the next part; As our data path is tested, the next test will concentrate on controller actually.

Top Module:

Top Module contains two parts: **CPU**, and **MEM**;
Although memory units was designed inside **Data Path**,
we put memory units in a separated module called **MEM**.

Top module is designed in schematic, available at:
"\$PATH/TO/PROJECT"/design/papers/

You can see the design here, too:



Top Modules' schematic diagram

Code:

Verilog Description of each part, and **Top Module** is available at: `“$PATH/TO/PROJECT”/code/top/`

Also a **test bench** is described to test **Top Module**, available at:

`“$PATH/TO/PROJECT”/code/test/`

To test its' functionality we have written the Bubble Sort algorithm, in assembly; The code has been at first written in simple **C**, then we convert it to Pseudo Instructions and then Pure Instructions; finally we encode the instructions into machine code, using our own Encoder, available in my github repository **“RV-Encoder”**; Also **C**, **assembly**, and **machine** codes are available at: `“$PATH/TO/PROJECT”/assembly/Sort/`

Simulation:

Finally, we have to examine **Top Module**.

We can check the correctness by data in memory elements; We have a set of ten **2's CMP, integer, 32bit Numbers** that must be sorted decreasingly. you can see the set order:

Data Memory before execution

Now we simulate the execution on “**Model Sim**”, by checking memory elements after execution, we can examine the functionality:

00000000	0001111100011110000000000000000000	000000001110100111000000001011101	00000000000000101111000100000000
00000003	0000000000000000001010101000000000	00000000000000000000000000000001101	1111110111000000000000000000001101
00000006	111010011111111111001111101010	101010101010100111111111111000	10101010000000000000000111111100
00000009	100111111011111101111000100000		
0000000c			

Data in memory cells (captured from "Model Sim")

[illegible]

Data in registers (captured from "Model Sim")

So we can approved our design functionality.

